

DueFlow

****DueFlow: 个人 DDL 信息自动化平台****

DueFlow 是一个面向学生、实习生和个人用户的自动化 Agent 系统。它把课程通知、邮件正文、PDF、Markdown、群公告和推送等非结构化信息统一收入 Inbox，再自动抽取任务、截止时间、提交物、风险点，并生成倒排计划和日历文件。

项目主线很明确：****把杂乱通知变成可执行的时间规划。****

解决的问题

日常通知里真正重要的信息通常分散在长文本中：

- 什么事情必须做
- 截止时间是什么
- 要提交哪些材料
- 通过什么方式提交
- 应该什么时候开始
- 哪些地方容易遗漏
- 是否需要加入日历提醒

DueFlow 不做通用聊天机器人，而是做一个个人信息自动化中枢。用户把信息丢进系统，系统自动完成整理、规划、提醒和导

目标用户

- 大学生和研究生
- 课程项目参与者
- 实习生和求职者
- 经常处理邮件、通知、比赛公告、项目安排的人
- 希望把个人任务管理自动化的人

核心功能

1. 统一 Inbox

所有输入都会被转换成统一的 `InboxItem`，记录来源、标题、原文、接收时间、处理状态和内容哈希。

优先实现：

- 手动粘贴文本
- 上传 `.txt`、`.md`、`.pdf`
- 扫描本地 `inbox/` 文件夹
- Webhook 接收外部系统推送

增强实现：

- IMAP 读取指定邮箱
- 邮件附件解析

2. 内容解析

系统将不同格式转换为纯文本：

- `.txt`：直接读取
- `.md`：按 Markdown 文本读取

- `.pdf`: 使用 `pdfplumber` 或 `pypdf` 提取文本
- Webhook: 读取 JSON 中的 `title`、`content`、`source`
- 邮件: 解析标题、正文、发件人和时间

3. LLM 结构化抽取

调用大模型 API，从原文中抽取：

- 任务名称
- 截止日期和时间
- 截止时间置信度
- 提交物
- 提交方式
- 地点或平台
- 优先级
- 缺失信息
- 原文依据

示例输出：

```
```json
{
 "title": "机器学习导论期末项目提交",
 "deadline": "2026-06-30 23:59",
 "deadline_confidence": "high",
 "deliverables": ["GitHub 开源代码", "README.md", "README PDF"],
 "submit_method": "课程平台提交",
 "priority": "high",
 "source_quote": "代码开源在 GitHub 或 Hugging Face, 编写 README.md 并转换成 PDF 提交",
 "missing_info": []
}
```

### ### 4. 日程规划 Agent

系统根据当前日期、DDL、任务复杂度和提交物数量，自动生成倒排计划。

示例：

- 6 月 18 日：确定选题和技术栈
- 6 月 20 日：完成基础功能
- 6 月 23 日：完成自动化接入
- 6 月 25 日：完成测试和 README
- 6 月 27 日：准备截图和演示
- 6 月 29 日：生成 README PDF 并最终检查
- 6 月 30 日：提交

### ### 5. 风险检查 Agent

系统主动发现计划中的问题：

- DDL 太近
- 截止时间不明确
- 缺少提交方式
- 提交物没有完全覆盖

- 多个任务集中到同一天
- 需要提前注册、报名或申请权限
- 计划中缺少最终检查

### ### 6. 导出与展示

必须支持：

- Streamlit 控制台展示 Inbox、任务、计划、风险
- 导出 `todo.md`
- 导出 `plan.md`
- 导出 `summary.md`
- 导出 `calendar.ics`
- 生成任务总结 Markdown

增强支持：

- Markdown 转 PDF
- 课程提交报告
- 按今天、本周、全部、风险任务筛选

## ## 系统架构

```

```text
多源输入
↓
统一 Inbox
↓
内容解析与去重
↓
LLM 结构化抽取
↓
SQLite 任务数据库
↓
日程规划 Agent
↓
风险检查 Agent
↓
Streamlit 控制台 / Markdown 导出 / ICS 导出
```

```

旁路接入：

```

```text
本地 inbox 文件夹 → 扫描器 → Inbox
Webhook → FastAPI → Inbox
文件上传 → Parser → Inbox
邮箱 IMAP → EmailConnector → Inbox
```

```

## ## 技术栈

- Python
- Streamlit: 个人控制台
- FastAPI: Webhook 接口

- SQLite: 本地持久化
- OpenAI-compatible SDK: 统一调用 Kimi、DeepSeek、智谱、Qwen
- `pdfplumber` 或 `pypdf`: PDF 解析
- `ics`: 生成日历文件
- `python-dotenv`: 管理 API Key
- `pytest`: 测试

## 快速开始

### ### 1. 创建并安装 conda 环境

推荐方式:

```
```bash
conda env create -f environment.yml
conda activate dueflow
```
```

手动方式:

```
```bash
conda create -y -n dueflow python=3.11 pip
conda activate dueflow
python -m pip install -r requirements.txt
```
```

本机固定路径方式:

```
```bash
conda create -y -p D:\conda_envs\dueflow python=3.11 pip
conda activate D:\conda_envs\dueflow
python -m pip install -r requirements.txt
```
```

如果 conda 提示 `libmamba` solver 不可用, 使用:

```
```bash
conda create -y --solver classic -p D:\conda_envs\dueflow python=3.11 pip
```
```

### ### 2. 配置环境变量

```
```bash
copy .env.example .env
```
```

默认 `LLM\_PROVIDER=mock`, 不需要 API Key 即可跑通演示。接入真实模型时, 修改 `.env`:

```
```env
LLM_PROVIDER=deepseek
LLM_API_KEY=your_api_key
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat
```
```

### ### 3. 运行测试

```
```bash
python -m pytest
```
```

本机固定路径:

```
```bash
D:\conda_envs\dueflow\python.exe -m pytest
```
```

### ### 4. 运行命令行演示

```
```bash
python scripts\run_demo.py
```
```

本机固定路径:

```
```bash
D:\conda_envs\dueflow\python.exe scripts\run_demo.py
```
```

该命令会读取 `examples/`, 生成:

- `exports/todo.md`
- `exports/plan.md`
- `exports/summary.md`
- `exports/calendar.ics`
- `exports/submission\_report.md`
- `dueflow\_demo.db`

### ### 5. 启动网页控制台

```
```bash
python -m streamlit run app.py
```
```

本机固定路径:

```
```bash
D:\conda_envs\dueflow\python.exe -m streamlit run app.py
```
```

### ### 6. 启动 Webhook API

```
```bash
python -m uvicorn api.webhook:app --reload
```
```

本机固定路径:

```
```bash
D:\conda_envs\dueflow\python.exe -m uvicorn api.webhook:app --reload
```

```
```
```

Webhook 示例：

```
```bash
curl -X POST http://127.0.0.1:8000/webhook/inbox ^
-H "Content-Type: application/json" ^
-d '{"title": "课程通知", "content": "请在 2026-06-30 23:59 前提交 README PDF。"}'
```
```

Webhook 支持自动处理，追加 `?process=true` 后会立即执行抽取、规划和风险检查：

```
```bash
curl -X POST "http://127.0.0.1:8000/webhook/inbox?process=true" ^
-H "Content-Type: application/json" ^
-d '{"title": "课程通知", "content": "请在 2026-06-30 23:59 前提交 README PDF。"}'
```
```

### 7. 导出 README PDF

```
```bash
python scripts\export_readme_pdf.py
```
```

本机固定路径：

```
```bash
D:\conda_envs\dueflow\python.exe scripts\export_readme_pdf.py
```
```

## 多模型设计

统一封装 `LLMProvider`，用相同接口调用不同模型。

推荐默认模型：

- DeepSeek：计划生成和推理
- Kimi：长文档理解
- 智谱 GLM：结构化抽取
- Qwen：备用模型或对比模型

实际复现时只要求配置一个可用 API Key。为了方便展示，系统必须提供 `mock` 模式，没有 API Key 也能跑通示例流程。

## 推荐目录结构

```
```text
DueFlow/
├── README.md
├── requirements.txt
├── .env.example
├── app.py
├── src/
│   ├── config.py
│   ├── database.py
│   └── models.py
```
```

```
| |— llm_provider.py
| |— parsers.py
| |— inbox.py
| |— extractors.py
| |— planner.py
| |— risk_checker.py
| |— calendar_exporter.py
| |— report_exporter.py
|— api/
| |— webhook.py
|— inbox/
|— examples/
| |— course_project_notice.md
| |— email_sample.txt
| |— competition_notice.md
|— exports/
|— docs/
| |— design.md
| |— superpowers/
| |— plans/
| |— 2026-06-16-dueflow-platform-implementation.md
|— tests/
...

```

## ## 开发阶段

第一阶段：完成可运行闭环。

- 项目结构、配置、数据库
- 手动输入和文件上传
- LLM 抽取或 mock 抽取
- 任务展示
- Markdown 导出

第二阶段：完成平台化能力。

- 本地 `inbox/` 扫描
- Webhook 接入
- PDF 解析
- 日程规划 Agent
- 风险检查 Agent
- ICS 导出

第三阶段：完成展示和复现。

- 示例数据
- Demo 截图
- README 完整启动说明
- 测试覆盖
- README 转 PDF

第四阶段：增强功能。

- IMAP 邮箱读取
- 按今天、本周、风险筛选

- 课程提交报告
- 多模型切换 UI

## ## 演示流程

1. 打开 Streamlit 控制台。
2. 粘贴机器学习导论课程项目说明。
3. 上传一个 PDF 通知。
4. 将一个`.md`文件放入`inbox/`, 点击扫描。
5. 使用 Webhook 模拟外部通知。
6. 查看统一 Inbox、抽取任务、倒排计划和风险提醒。
7. 导出`todo.md`、`plan.md`、`summary.md`、`calendar.ics`、`submission\_report.md`。
8. 展示 README、示例数据和复现命令。

## ## 展示材料

建议在`docs/images/`中补充以下截图后再提交课程作业：

- `01-dashboard.png`：首页指标和任务概览
- `02-inbox.png`：文本粘贴、文件上传或本地扫描入口
- `03-tasks.png`：任务、计划和风险检查结果
- `04-exports.png`：Markdown、ICS 和提交报告导出结果

命令行演示已生成示例输出到`exports/`, 这些文件可作为 GitHub 仓库中的可复现样例。

## ## 课程评分匹配

- 调用大模型 API：通过统一`LLMProvider`调用 Kimi、DeepSeek、智谱或 Qwen。
- Agent 应用：包含结构化抽取、日程规划、风险检查三个 Agent 能力。
- 实用性：解决 DDL 遗漏、任务拆解和提醒管理问题。
- 可展示：Streamlit 页面直接展示输入、任务、计划、风险和导出结果。
- 可复现：提供 mock 模式、示例数据、`.env.example`、启动命令和测试。
- 可开源：项目结构清晰，适合发布到 GitHub 或 Hugging Face。

## ## 开源复现检查

发布到 GitHub 前确认：

- 已提交`README.md`、`README.pdf`、`environment.yml`、`.env.example`、`LICENSE`和测试代码。
- 不提交`.env`、`.db`、`.pytest\_cache/`、`\_\_pycache\_\_/`等运行时文件。
- 使用`python -m pytest`能通过全部测试。
- 使用`python scripts/run\_demo.py`能重新生成`exports/`示例结果。
- 真实大模型 API Key 只写入本地`.env`，不要写入 README、代码或测试。

## ## License

This project is released under the MIT License. See `LICENSE` for details.

## ## 不做的内容

- 不做账号系统
- 不做多人协作
- 不自动读取微信或 QQ
- 不做移动端 App
- 不直接写入 Google Calendar 或 Outlook



- 不做复杂工作流编排平台

这些内容只作为未来扩展，不影响本课程项目交付。